Basic (your first working tool) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `first_tool_call_practice.py`: this is the exact primitive every agent in this entire curriculum is built from: you bind a tool to the model with `.bind_tools([...])`, and the model decides — on its own — to *request* a function call (which comes back in `response.tool_calls`) instead of answering in text; your code then runs that function and could feed the result back. Seeing this one round trip work standalone, before combining it with anything else, is what makes the Build Task's `run_with_tools()` loop obvious instead of magic. **If not:** the first time you'd see a tool call requested and executed would be inside a bigger harness already juggling 2-3 tools, making it harder to tell which part is "the round trip" and which part is everything else.

Solution

```
# first_tool_call_practice.py
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI


@tool
def add(a: int, b: int) -> int:
    """Add two integers together and return the sum."""
    return a + b


# .bind_tools() gives back a model that can choose to call our tool.
model = ChatOpenAI(model="gpt-4o-mini", temperature=0).bind_tools([add])
response = model.invoke("What's 5 + 7?")

# Even for obvious math, the model might just answer in text — always check.
if not response.tool_calls:
    print("The model answered directly, without calling a tool.")
    print(response.content)
else:
    for call in response.tool_calls:
        if call["name"] == "add":
            # .invoke() on the tool — @tool wraps add into a Runnable too
            result = add.invoke(call["args"])
            print(f"Called add({call['args']}) -> {result}")
```

```
        else:
            print(f"Unexpected tool requested: {call['name']}")
```

Expected output:

```
Called add({'a': 5, 'b': 7}) -> 12
```